

## 5.1 LA DATA SCIENCE COMMENCE DANS VOTRE BASE DE DONNÉES

La data science n'est pas réservée aux grands laboratoires avec des serveurs de calcul. Dans une capsule Gykhamine, votre base SQLite contient déjà de la donnée exploitable dès le premier mois d'usage.

LES 3 NIVEAUX D'ANALYSE SELON LE CONTEXTE :

- NIVEAU 1 – DESCRIPTIF : "Qu'est-ce qui s'est passé ?"
- Rapports, totaux, moyennes, tableaux de bord
  - Outils : Django ORM, pandas, matplotlib
- NIVEAU 2 – DIAGNOSTIQUE : "Pourquoi est-ce arrivé ?"
- Corrélations, comparaisons, détection d'anomalies
  - Outils : pandas, scipy, seaborn
- NIVEAU 3 – PRÉDICTIF : "Qu'est-ce qui va se passer ?"
- Régression, classification, prévisions de ventes
  - Outils : scikit-learn, statsmodels

Dans une capsule, vous commencez au Niveau 1 et montez progressivement selon le besoin.

## 5.2 ANALYSER LES DONNÉES DEPUIS DJANGO – SANS BIBLIOTHÈQUE EXTERNE

Django ORM fournit Sum, Avg, Count, Min, Max nativement.

EXEMPLE – Rapport mensuel de ventes (Boutique ERP) :

```
from django.db.models import Sum, Avg, Count
from django.db.models.functions import TruncMonth
from .models import Vente

def rapport_mensuel(request):
    # Agréger les ventes par mois
    ventes_par_mois = (
        Vente.objects
        .annotate(mois=TruncMonth('date_vente'))
        .values('mois')
        .annotate(
            total_fcfa = Sum('montant_total'),
            nb_ventes = Count('id'),
            panier_moyen = Avg('montant_total'),
        )
        .order_by('mois')
    )

    return render(request, 'rapports/mensuel.html', {
        'ventes_par_mois': ventes_par_mois
    })
```

RÉSULTAT SOUS FORME DE TABLE :

| Mois     | Total FCFA | Nb Ventes | Panier Moyen |
|----------|------------|-----------|--------------|
| Jan 2025 | 4 580 000  | 312       | 14 679       |
| Fév 2025 | 3 920 000  | 278       | 14 101       |
| Mar 2025 | 5 210 000  | 401       | 12 993       |

## 5.3 EXPORTER VERS PANDAS POUR ANALYSE AVANCÉE

DANS UNE VUE OU UN SCRIPT DE RAPPORT :

```
import pandas as pd
from .models import Vente

def analyser_ventes_pandas():
    # Convertir le QuerySet Django en DataFrame pandas
    qs = Vente.objects.values('date_vente', 'montant_total', 'produit__categorie__nom')
    df = pd.DataFrame(list(qs))

    # Renommer les colonnes pour la lisibilité
    df.columns = ['date', 'montant', 'categorie']
    df['date'] = pd.to_datetime(df['date'])
    df['mois'] = df['date'].dt.to_period('M')

    # Analyse par catégorie
    analyse = df.groupby('categorie')['montant'].agg(['sum', 'mean', 'count'])
    analyse.columns = ['Total FCFA', 'Moyenne FCFA', 'Nb Transactions']
    analyse = analyse.sort_values('Total FCFA', ascending=False)

    return analyse
```

SORTIE ATTENDUE :

| categorie    | Total FCFA | Moyenne FCFA | Nb Transactions |
|--------------|------------|--------------|-----------------|
| Alimentation | 12 450 000 | 8 500        | 1465            |
| Hygiène      | 4 230 000  | 6 200        | 682             |
| Boissons     | 3 180 000  | 5 400        | 589             |

---

## 5.4 PRÉVISION SIMPLE – RÉGRESSION LINÉAIRE SUR LES VENTES

---

Pour prévoir les ventes du mois prochain à partir des données passées :

```
import numpy as np
from sklearn.linear_model import LinearRegression

def prevoir_ventes_prochains_mois(df_mensuel):
    """
    df_mensuel : DataFrame avec colonnes ['mois_num', 'total_fcfa']
    mois_num : 1, 2, 3, 4... (nombre de mois depuis le début)
    """
    X = df_mensuel[['mois_num']].values
    y = df_mensuel['total_fcfa'].values

    modele = LinearRegression()
    modele.fit(X, y)

    # Prévision pour le mois suivant
    prochain_mois = [[len(df_mensuel) + 1]]
    prevision = modele.predict(prochain_mois)[0]

    return {
        'prevision_fcfa': round(prevision),
        'tendance': 'hausse' if modele.coef_[0] > 0 else 'baisse',
        'variation_mois': round(modele.coef_[0]),
    }
```

INTERPRÉTATION :

prevision\_fcfa → Montant prévu pour le mois suivant  
 tendance → "hausse" ou "baisse" globale  
 variation\_mois → De combien augmente/diminue chaque mois en FCFA

---

## 5.5 INTÉGRER L'ANALYSE DANS UN TABLEAU DE BORD DJANGO

---

Vue intégrant ORM + pandas + prévision :

```
def dashboard_analytique(request):
    analyse = analyser_ventes_pandas()
    df_mensuel = construire_df_mensuel()
    prevision = prevoir_ventes_prochains_mois(df_mensuel)
    top_produits = Produit.objects.annotate(
        ventes_totales=Sum('lignedvente__quantite')
    ).order_by('-ventes_totales')[:5]
```

```
return render(request, 'dashboard_analytique.html', {
    'analyse_categorie' : analyse.to_dict('records'),
    'prevision'         : prevision,
    'top_produits'      : top_produits,
})
```

Cette vue donne un tableau de bord complet utilisable dans n'importe quel contexte : boutique, pharmacie, microfinance, logistique.

---

## RÉSUMÉ DU CHAPITRE 5

---

- ✓ 3 niveaux : descriptif → diagnostique → prédictif
- ✓ Django ORM (Sum, Avg, Count) = analyse Niveau 1 sans bibliothèque
- ✓ Pandas = passerelle vers l'analyse avancée (pd.DataFrame(list(qs)))
- ✓ Régression linéaire = prévision en 10 lignes avec scikit-learn
- ✓ Tout s'intègre dans une vue Django standard avec render()

→ SUITE : Chapitre 6 – Le Paris Sportif comme cas d'étude Data Science